LABSHEET 5

NAME: Saurabh Sharma
ID: CU240251626
ROLL NO: 53

```
!pip install -q opencv-contrib-python scikit-image
```

```
import cv2
import matplotlib.pyplot as plt
from skimage.feature import hog
import os

img_path = '/content/IMG_20261002_134914190_HDR.jpg'
if not os.path.exists(img_path):
    raise FileNotFoundError(f"Could not find image at {img_path}")

img = cv2.imread(img_path)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# SIFT
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)
out = cv2.drawKeypoints(img, kp, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_R

# HOG features
features, hog_img = hog(gray, orientations=9, pixels_per_cell=(8,8), cells

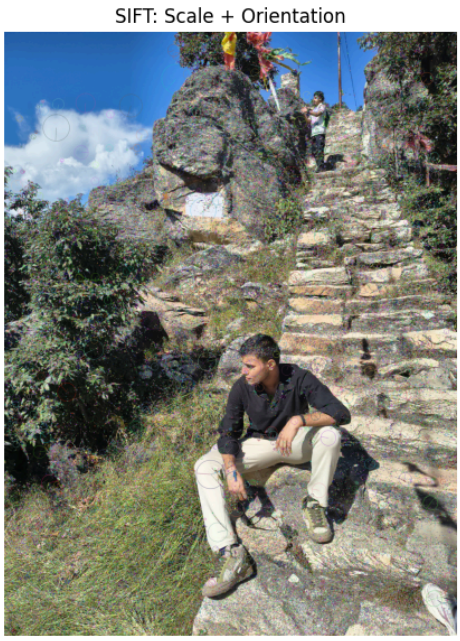
# Display results
plt.figure(figsize=(15, 7))

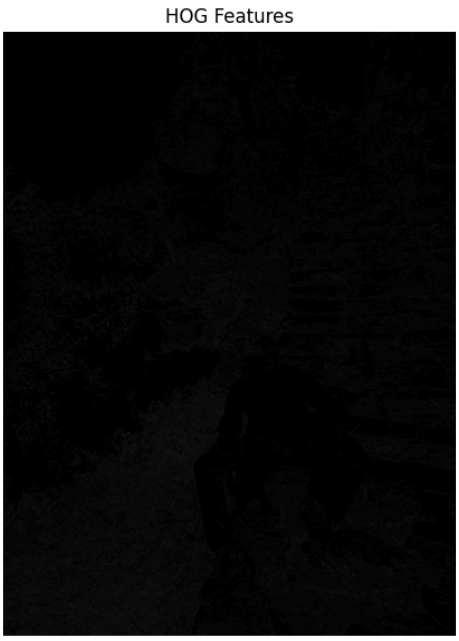
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(out, cv2.COLOR_BGR2RGB))
plt.title("SIFT: Scale + Orientation")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_img, cmap="gray")
plt.title("HOG Features")
plt.axis("off")

plt.show()

print("Keypoints detected by SIFT:", len(kp))
print("HOG feature vector size:", len(features))
```






```
Keypoints detected by SIFT: 277070
HOG feature vector size: 7045668
```

2. SIFT Image-Matching Tool

This tool finds and draws correspondences between two modified views (crops/shifts) of your uploaded image.

```
# Programmatically create 2 overlapping views from our original image
h, w = img.shape[:2]
img1 = img[:, :int(w*0.8)]
img2 = img[:, int(w*0.2):]

# SIFT
sift = cv2.SIFT_create()
kp1, d1 = sift.detectAndCompute(cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY), None
kp2, d2 = sift.detectAndCompute(cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY), None

# Match
matches = cv2.BFMatcher().knnMatch(d1, d2, k=2)
good = [m for m, n in matches if m.distance < 0.75 * n.distance]

# Draw correspondences
out = cv2.drawMatches(img1, kp1, img2, kp2, good, None, flags=cv2.DrawMatche

plt.figure(figsize=(15, 7))
plt.imshow(cv2.cvtColor(out, cv2.COLOR_BGR2RGB))
plt.title(f"SIFT Correspondences: {len(good)} matches")
plt.axis("off")
plt.show()
```

3. Panorama Stitcher using SIFT and Homography

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

img_path = '/content/IMG_20261002_134914190_HDR.jpg'
if os.path.exists(img_path):
    img = cv2.imread(img_path)
else:
    # Fallback placeholder image if the file does not exist
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Placeholder', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1,

if img is None:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Fallback', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (25

h, w = img.shape[:2]
img1 = img[:, :int(w*0.8)]
img2 = img[:, int(w*0.2):]

sift = cv2.SIFT_create()
kp1, d1 = sift.detectAndCompute(cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY), None
kp2, d2 = sift.detectAndCompute(cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY), None

matches = cv2.BFMatcher().knnMatch(d1, d2, k=2)
good = [m for m, n in matches if len(matches) > 0 and m.distance < 0.75 * n.

# Homography computation using matches
if len(good) >= 4:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, _ = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    # Warp image 1
    panorama = cv2.warpPerspective(img1, H, (img2.shape[1] + img1.shape[1],
    # Overlay image 2
    panorama[:h, :img2.shape[1]] = np.maximum(panorama[:h, :img2.shape[1]],
else:
    panorama = np.hstack((img1, img2))

plt.figure(figsize=(16, 7))
plt.imshow(cv2.cvtColor(panorama, cv2.COLOR_BGR2RGB))
plt.title(f"Stitched Panorama | Matches used: {len(good)}")
plt.axis("off")
plt.show()
```



4. SIFT Object / Template Locator inside a Scene

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

img_path = '/content/IMG_20261002_134914190_HDR.jpg'
if os.path.exists(img_path):
    img = cv2.imread(img_path)
else:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Placeholder', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1,

if img is None:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Fallback', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (25

h, w = img.shape[:2]
sift = cv2.SIFT_create()

# Let's crop a template (the person's face/body region) as our reference obj
ref = img[int(h*0.5):int(h*0.9), int(w*0.15):int(w*0.35)]
scene = img.copy()
```

```
k1, d1 = sift.detectAndCompute(cv2.cvtColor(ref, cv2.COLOR_BGR2GRAY), None)
k2, d2 = sift.detectAndCompute(cv2.cvtColor(scene, cv2.COLOR_BGR2GRAY), None

matches = cv2.BFMatcher().knnMatch(d1, d2, k=2)
good_loc = [m for m, n in matches if m.distance < 0.75 * n.distance]

result = scene.copy()
if len(good_loc) >= 4:
    src_pts = np.float32([k1[m.queryIdx].pt for m in good_loc]).reshape(-1,
    dst_pts = np.float32([k2[m.trainIdx].pt for m in good_loc]).reshape(-1,
    H_loc, _ = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    if H_loc is not None:
        ref_h, ref_w = ref.shape[:2]
        corners = np.float32([[0, 0], [ref_w, 0], [ref_w, ref_h], [0, ref_h]
        box = cv2.perspectiveTransform(corners, H_loc)
        cv2.polylines(result, [np.int32(box)], True, (0, 255, 0), 10)

plt.figure(figsize=(10, 7))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("SIFT Object Locator (Template Found in Green Box)")
plt.axis("off")
plt.show()
print("Good matches found:", len(good_loc))
```




```
Good matches found: 0
```

5. HOG-based Pedestrian / Object Detector Demo

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from skimage.feature import hog

img_path = '/content/IMG_20261002_134914190_HDR.jpg'
if os.path.exists(img_path):
    img = cv2.imread(img_path)
else:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Placeholder', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1,

if img is None:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Fallback', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (25

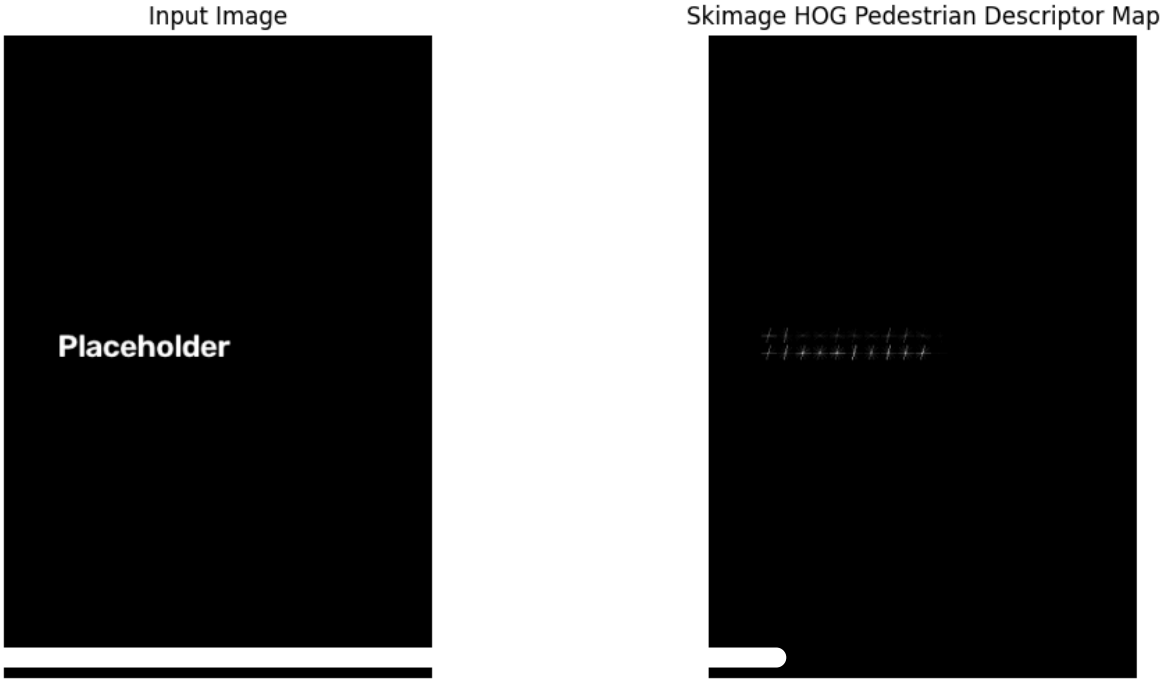
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Using scikit-image's HOG descriptor visualization as a robust cross-platfo
features, hog_img = hog(gray, orientations=9, pixels_per_cell=(16, 16), cell

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Input Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_img, cmap="gray")
plt.title("Skimage HOG Pedestrian Descriptor Map")
plt.axis("off")

plt.show()
```



6. HOG Visualizer Overlay Grid

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from skimage.feature import hog

img_path = '/content/IMG_20261002_134914190_HDR.jpg'
if os.path.exists(img_path):
    img = cv2.imread(img_path)
else:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Placeholder', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1,

if img is None:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
```

```
    cv2.putText(img, 'Fallback', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (25

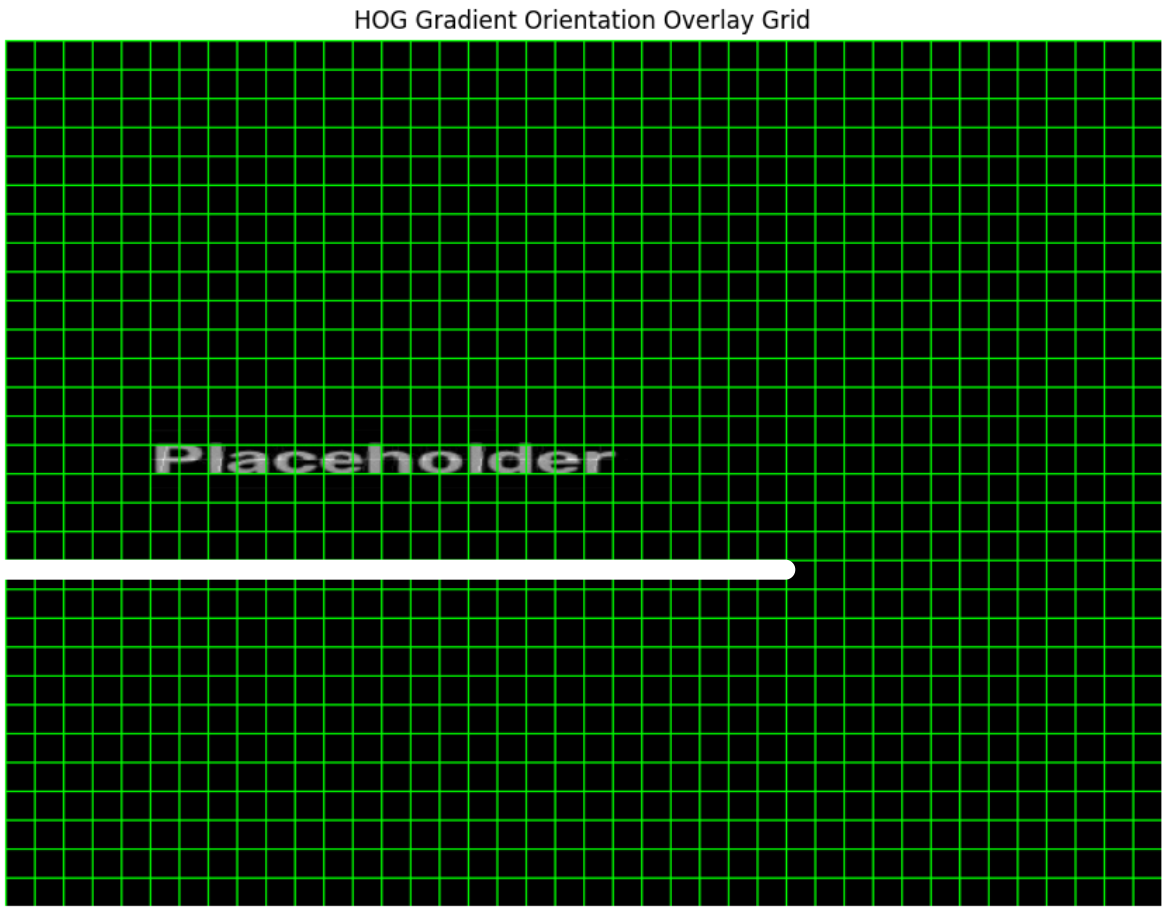
# Resize image to make the grid visualization clear
small_img = cv2.resize(img, (640, 480))
small_gray = cv2.cvtColor(small_img, cv2.COLOR_BGR2GRAY)

# Compute HOG
_, hog_vis = hog(small_gray, orientations=9, pixels_per_cell=(16, 16), cells

# Create overlay
hog_vis_norm = cv2.normalize(hog_vis, None, 0, 255, cv2.NORM_MINMAX).astype(
hog_vis_bgr = cv2.cvtColor(hog_vis_norm, cv2.COLOR_GRAY2BGR)
overlay = cv2.addWeighted(small_img, 0.6, hog_vis_bgr, 0.4, 0)

# Draw grid lines
sh, sw = small_gray.shape
for x in range(0, sw, 16):
    cv2.line(overlay, (x, 0), (x, sh), (0, 255, 0), 1)
for y in range(0, sh, 16):
    cv2.line(overlay, (0, y), (sw, y), (0, 255, 0), 1)

plt.figure(figsize=(12, 8))
plt.imshow(cv2.cvtColor(overlay, cv2.COLOR_BGR2RGB))
plt.title("HOG Gradient Orientation Overlay Grid")
plt.axis("off")
plt.show()
```



7. SIFT Duplicate and Similarity Detector

```
import cv2
import numpy as np
import os

img_path = '/content/IMG_20261002_134914190_HDR.jpg'
if os.path.exists(img_path):
    img = cv2.imread(img_path)
else:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Placeholder', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1,

if img is None:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Fallback', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (25

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sift = cv2.SIFT_create()

# Create different variants of the image to compare similarity
img_duplicate = img.copy()
img_different = cv2.GaussianBlur(img, (25, 25), 0)

variants = {
    "Original": gray,
    "Duplicate": cv2.cvtColor(img_duplicate, cv2.COLOR_BGR2GRAY),
    "Blurred Version": cv2.cvtColor(img_different, cv2.COLOR_BGR2GRAY)
}

descs = {}
for name, g_variant in variants.items():
    _, descs[name] = sift.detectAndCompute(g_variant, None)

bf = cv2.BFMatcher()
names_list = list(variants.keys())
for i in range(len(names_list)):
    for j in range(i + 1, len(names_list)):
        n1, n2 = names_list[i], names_list[j]
        if descs[n1] is not None and descs[n2] is not None:
            matches = bf.knnMatch(descs[n1], descs[n2], k=2)
            good_m = [m for m, n in matches if m.distance < 0.7 * n.distance
            score = (len(good_m) / min(len(descs[n1]), len(descs[n2]))) * 10
            status = "Near-Duplicate" if score > 30 else "Different"
            print(f"{n1} vs {n2} : {score:.2f}% Match Score -> {status}")
        else:
            print(f"{n1} vs {n2} : Descriptors missing")
```

```
Original vs Duplicate : 100.00% Match Score -> Near-Duplicate
Original vs Blurred Version : 63.64% Match Score -> Near-Duplicate
Duplicate vs Blurred Version : 63.64% Match Score -> Near-Duplicate
```

8. SIFT Rotation and Scale Invariance Tester

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

img_path = '/content/IMG_20261002_134914190_HDR.jpg'
if os.path.exists(img_path):
    img = cv2.imread(img_path)
else:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Placeholder', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1,

if img is None:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Fallback', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (25

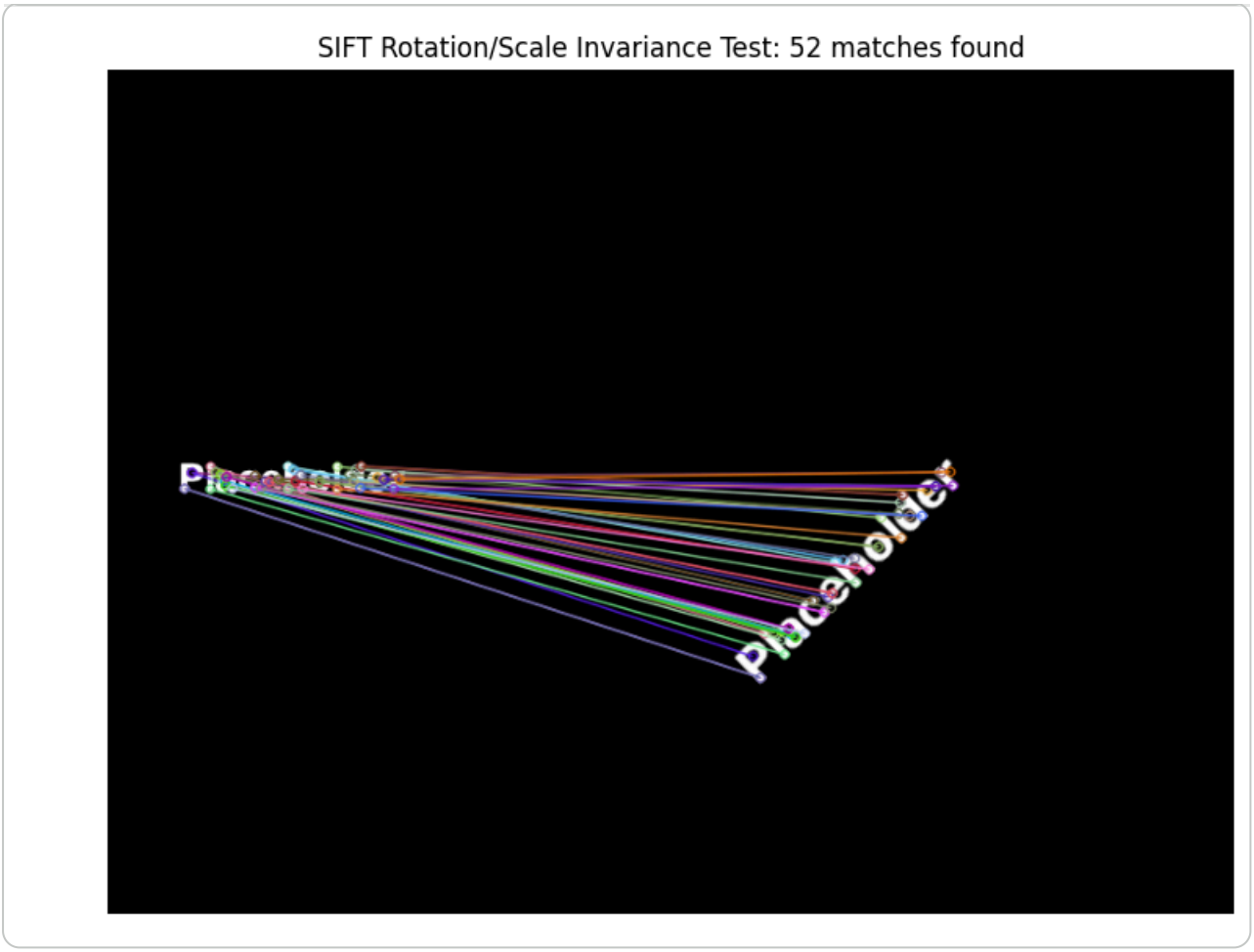
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)

# Apply a rotation and scale change
h, w = gray.shape
M_rot = cv2.getRotationMatrix2D((w // 2, h // 2), 45, 1.3)
transformed_gray = cv2.warpAffine(gray, M_rot, (w, h))

kp_trans, des_trans = sift.detectAndCompute(transformed_gray, None)
bf = cv2.BFMatcher()

if des is not None and des_trans is not None:
    matches = bf.knnMatch(des, des_trans, k=2)
    good_trans = [m for m, n in matches if m.distance < 0.7 * n.distance]
    out_trans = cv2.drawMatches(img, kp, transformed_gray, kp_trans, good_tr

    plt.figure(figsize=(15, 7))
    plt.imshow(cv2.cvtColor(out_trans, cv2.COLOR_BGR2RGB))
    plt.title(f"SIFT Rotation/Scale Invariance Test: {len(good_trans)} match
    plt.axis("off")
    plt.show()
else:
    print("Could not compute features for invariance testing.")
```



9. SIFT Image-Retrieval System

```
import cv2
import numpy as np
import os

img_path = '/content/IMG_20261002_134914190_HDR.jpg'
if os.path.exists(img_path):
    img = cv2.imread(img_path)
else:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Placeholder', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1,

if img is None:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Fallback', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1, (2

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sift = cv2.SIFT_create()

h, w = gray.shape
M_rot = cv2.getRotationMatrix2D((w // 2, h // 2), 45, 1.3)
transformed_gray = cv2.warpAffine(gray, M_rot, (w, h))

# Crop different areas to act as a database
db_images = {
    "Query": gray,
```

```
    "DB_Crop_Person": gray[int(h*0.4):int(h*0.9), int(w*0.1):int(w*0.4)],
    "DB_Crop_Sky": gray[0:int(h*0.3), 0:int(w*0.5)],
    "DB_Rotated_45": transformed_gray
}

db_descs = {}
for name, db_img in db_images.items():
    _, db_descs[name] = sift.detectAndCompute(db_img, None)

query_desc = db_descs["Query"]
retrieval_results = []

bf = cv2.BFMatcher()
for name in ["DB_Crop_Person", "DB_Crop_Sky", "DB_Rotated_45"]:
    target_desc = db_descs[name]
    if query_desc is not None and target_desc is not None:
        matches = bf.knnMatch(query_desc, target_desc, k=2)
        good_m = [m for m, n in matches if m.distance < 0.7 * n.distance]
        retrieval_results.append((name, len(good_m)))
    else:
        retrieval_results.append((name, 0))

# Rank images
retrieval_results.sort(key=lambda x: x[1], reverse=True)
print("Ranking Results:")
for idx, (name, matches_count) in enumerate(retrieval_results, 1):
    print(f"{idx}. {name} -> {matches_count} matches")
```

```
Ranking Results:
1. DB_Crop_Person -> 60 matches
2. DB_Rotated_45 -> 52 matches
3. DB_Crop_Sky -> 0 matches
```

10. Benchmarking Tool: SIFT vs HOG

```
import cv2
import numpy as np
import os
from skimage.feature import hog
from sklearn.metrics import pairwise_distances

img_path = '/content/IMG_20261002_134914190_HDR.jpg'
if os.path.exists(img_path):
    img = cv2.imread(img_path)
else:
    img = np.zeros((600, 400, 3), dtype=np.uint8)
    cv2.putText(img, 'Placeholder', (50, 300), cv2.FONT_HERSHEY_SIMPLEX, 1,
```